

```
pip install pandas numpy tldextract
```

```
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
Requirement already satisfied: tldextract in /usr/local/lib/python3.12/dist-packages (5.3.1)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2026.2)
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages (from tldextract) (3.18)
Requirement already satisfied: requests>=2.1.0 in /usr/local/lib/python3.12/dist-packages (from tldextract) (2.32.4)
Requirement already satisfied: requests-file>=1.4 in /usr/local/lib/python3.12/dist-packages (from tldextract) (3.0.1)
Requirement already satisfied: filelock>=3.0.8 in /usr/local/lib/python3.12/dist-packages (from tldextract) (3.29.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.1.0->tldextract) (3.4.1)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.1.0->tldextract) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests>=2.1.0->tldextract) (2026.5.20)
```

```
import re
import pandas as pd
import tldextract
import random
print("Loading datasets...")
# Load phishing URLs
df_phish = pd.read_csv("/content/verified_online.csv.gz")
phish_urls = df_phish["url"].dropna().tolist()[:40000]
# Load benign URLs
df_tranco = pd.read_csv(
    "tranco_XW5QN.csv",
    header=None,
    names=["rank", "domain"]
)
tranco_urls = ("https://" + df_tranco["domain"].astype(str)).tolist()
# New dataset
df_new = pd.read_csv("URL_dataset.csv")
df_new = df_new[df_new["type"].str.lower() == "legitimate"]
df_new["url"] = df_new["url"].str.replace("://www.", "://" , regex=False)
new_urls = df_new["url"].dropna().tolist()
# Merge and remove duplicates
benign_urls = list(set(tranco_urls + new_urls))
random.seed(42)
random.shuffle(benign_urls)
benign_urls = benign_urls[:40000]
print(len(benign_urls))
print(f"Loaded {len(phish_urls)} phishing URLs and {len(benign_urls)} benign URLs.")
```

```
Loading datasets...
40000
Loaded 40000 phishing URLs and 40000 benign URLs.
```

```
print(benign_urls[:10])
```

```
['https://natashashome.com', 'https://agua.glass', 'https://afiki-t.com', 'https://iv3a3im2.com', 'https://argmn.com', 'https://lucktech.co
```

```
from collections import Counter
import re
import math
from urllib.parse import urlparse
def extract_feat(url,lbl):
    feat = {"url": url , "label" : lbl}
    feat["url_length"] = len(url)
    feat["num_dots"] = url.count('.')
    feat["num_digits"] = sum(c.isdigit() for c in url)
    feat["digit_ratio"] = feat["num_digits"]/len(url)
    # feat["has_@"] = 1 if "@" in url else 0
    ip_pattern = r"https?://\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}"
    feat["has_ip"] = 1 if re.match(ip_pattern,url) else 0
    ext = tldextract.extract(url)
    dom = ext.domain
    sub_dom = ext.subdomain
    # feat["num_sub_dom"] = len(sub_dom.split(".")) if sub_dom else 0
    feat["num_delimiters_dom"] = dom.count("-") + dom.count("_")
    parsed = urlparse(url)
    path = parsed.path
    feat["num_delimiters_path"] = path.count("/") + path.count("-")
    # feat["subdomain_depth"] = sub_dom.count(".") + 1 if sub_dom else 0
    keywords = ["login","signin","verify","secure","account","update","confirm","password","wallet","bank","payment","paypal","invoice","gift"]
    feat["has_key_wrd"] = (1 if any(ky in url.lower() for ky in keywords) else 0)
    tld_phish = ["tk","ml","cf","gq","ga","bond","win","cfd","help","shop","buzz","li","es"]
    feat["phish_TLD"] = (1 if any(t in ext.suffix for t in tld_phish) else 0)
    feat["subtle_misspell"] = misspell_score(dom)
    feat["dom"] = dom
    feat["dom_entropy"] = cal_ana_entropy(dom)
    feat["path_entropy"] = cal_ana_entropy(path)
    tokens = re.split(r'[\-\_\.\?=&]', url)
    feat["num_tokens"] = len([t for t in tokens if t])
    feat["max_token_len"] = max((len(t) for t in tokens if t),default=0)
    # print(ext.suffix)
    return feat
```

```

def lcs(a,b):
    m = len(a)
    n = len(b)
    if m <= 3:
        return 0
    dp = [[0]*(n+1) for _ in range(m+1)]
    ans = 0
    for i in range(1,m+1):
        for j in range(1,n+1):
            if a[i-1] == b[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
                ans = max(ans,dp[i][j])
    return ans

def misspell_score(dom):
    # don't flag if domain is the brand exactly
    brands = ["google", "paypal", "amazon", "facebook", "microsoft",
              "apple", "netflix", "instagram", "twitter", "linkedin",
              "dropbox", "whatsapp", "youtube", "gmail", "outlook", "chatgpt", "reddit"]
    if dom.lower() in brands:
        return 0.0
    mx = 0
    for b in brands:
        comm = lcs(dom, b)
        score = comm / max(len(b), len(dom)) # normalizing against both lengths
        mx = max(mx, score)
    return mx

def cal_ana_entropy(dom):
    if not dom:
        return 0
    dom_len = len(dom)
    prob = [count/dom_len for count in Counter(dom).values()]
    entropy = -sum(p*math.log(p)/math.log(2) for p in prob)
    # is_sus = (round(entropy,3) > 3.5 and len(dom)>6)
    return entropy

dataset_rows = []
for ur in phish_urls:
    dataset_rows.append(extract_feat(ur,1))
for ur in benign_urls:
    dataset_rows.append(extract_feat(ur,0))

df = pd.DataFrame(dataset_rows)
df.to_csv("phishing_dataset.csv",index=False)
print(df.tail())

```

```

      url  label  url_length  num_dots  num_digits  \
79995  https://malliq-api.com      0         22         1         0
79996  https://ltjuy4xb.com      0         20         1         1
79997  https://orderfg.top      0         19         1         0
79998  https://lodgerunner.com      0         23         1         0
79999  https://acul.com      0         16         1         0

  digit_ratio  has_ip  num_delimiters_dom  num_delimiters_path  \
79995      0.00      0              1              0
79996      0.05      0              0              0
79997      0.00      0              0              0
79998      0.00      0              0              0
79999      0.00      0              0              0

  has_key_wrd  phish_TLD  subtle_misspell      dom  dom_entropy  \
79995      0          0      0.200000  malliq-api  2.721928
79996      0          0      0.125000  ltjuy4xb  3.000000
79997      0          0      0.285714  orderfg  2.521641
79998      0          0      0.181818  lodgerunner  2.913977
79999      0          0      0.250000  acul  2.000000

  path_entropy  num_tokens  max_token_len
79995      0.0          4          6
79996      0.0          3          8
79997      0.0          3          7
79998      0.0          3         11
79999      0.0          3          6

```

```

# Check how feature distributions differ between Safe (0) and Phishing (1)
print(df.groupby("label").describe())

```

```

      url_length  count      mean      std  min  25%  50%  75%  max  \
label
0      40000.0  24.491350  11.064549  12.0  19.0  22.0  26.0  307.0
1      40000.0  50.149675  56.539852  15.0  23.0  36.0  48.0  2079.0

      num_dots  count      mean  ...  num_tokens  max_token_len  \
label
0      40000.0  1.1209  ...      4.0  47.0      40000.0  9.877150
1      40000.0  1.8825  ...      7.0  171.0      40000.0  16.141375

      std  min  25%  50%  75%  max
label
0      4.094719  6.0  6.0  9.0  12.0  90.0
1      24.732778  5.0  6.0  9.0  16.0  1015.0

```

```

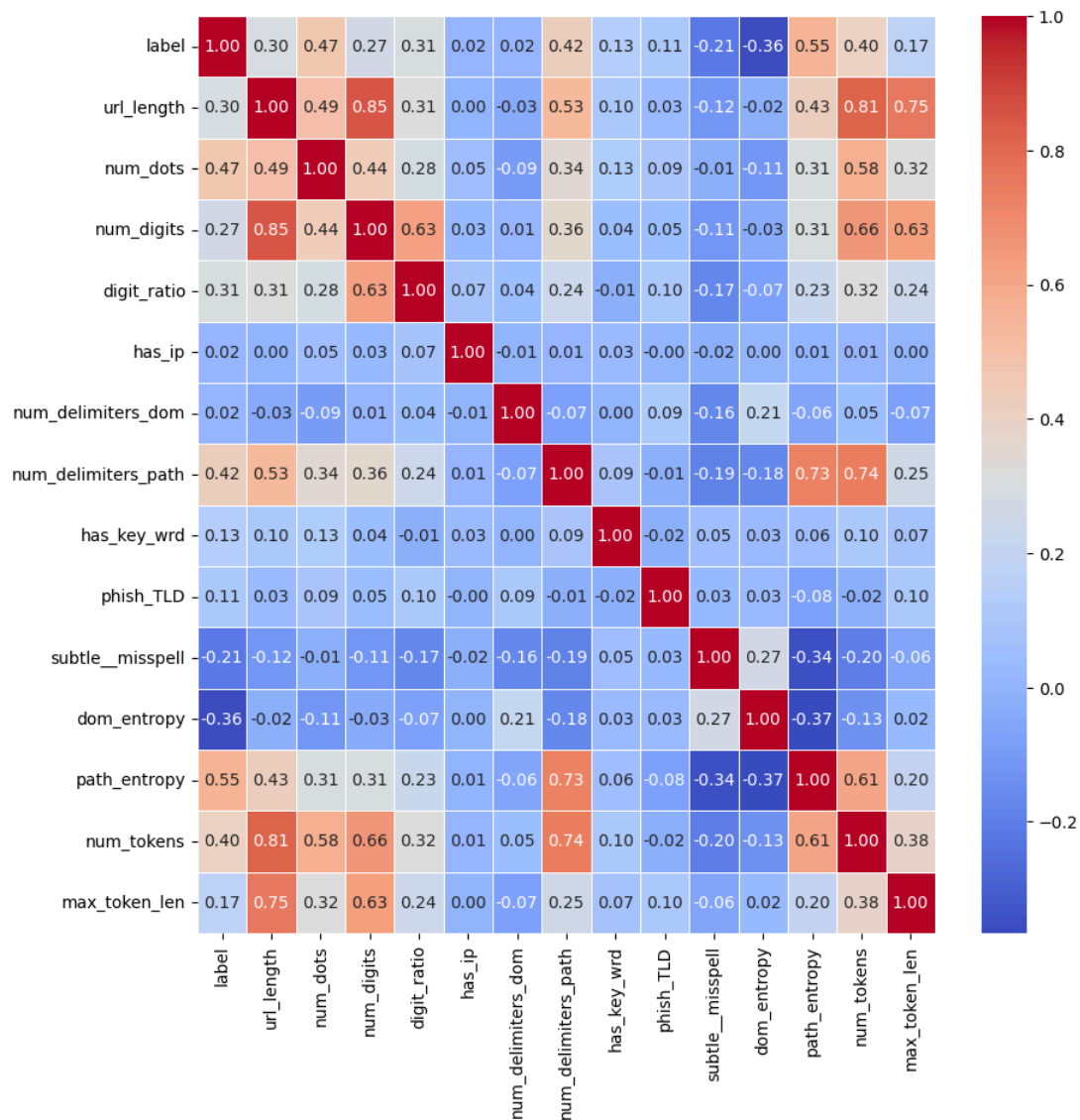
[2 rows x 112 columns]

```

```

import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
df = pd.read_csv("phishing_dataset.csv")
correlation = df.drop(["url", "dom"], axis=1).corr()
plt.figure(figsize=(10,10))
sns.heatmap(correlation, annot=True, cmap="coolwarm", fmt = ".2f" , linewidths = 0.5)
plt.show()

```



```

df_safe = df[df["label"] == 0]
df_phish = df[df["label"] == 1]
print("Printing safe stats")
print(df_safe.describe())
print("Printing phishing stats")
print(df_phish.describe())

```

	count	40000.000000	40000.000000	40000.000000	40000.000000
mean	0.032750	0.208498	2.803764	0.206425	
std	0.177984	0.088966	0.525123	0.868224	
min	0.000000	0.000000	-0.000000	0.000000	

```

75%      1.0      48.000000      2.000000      0.000000      0.102304
max       1.0     2079.000000      67.000000     333.000000      0.676301

      has_ip  num_delimiters_dom  num_delimiters_path  has_key_wrd  \
count  40000.000000      40000.000000      40000.000000  40000.000000
mean    0.000500      0.183075      1.677300      0.051975
std     0.022355      0.436249      1.774301      0.221979
min     0.000000      0.000000      0.000000      0.000000
25%     0.000000      0.000000      1.000000      0.000000
50%     0.000000      0.000000      1.000000      0.000000
75%     0.000000      0.000000      2.000000      0.000000
max      1.000000      7.000000     29.000000      1.000000

      phish_TLD  subtle_misspell  dom_entropy  path_entropy  \
count  40000.000000      40000.000000  40000.000000  40000.000000
mean    0.083550      0.161849      2.347298      2.016960
std     0.276715      0.122038      0.653839      1.713250
min     0.000000      0.000000     -0.000000     -0.000000
25%     0.000000      0.000000      1.918296     -0.000000
50%     0.000000      0.166667      2.251629      2.521641
75%     0.000000      0.250000      2.913977      3.000000
max      1.000000      0.888889      4.323231      5.960668

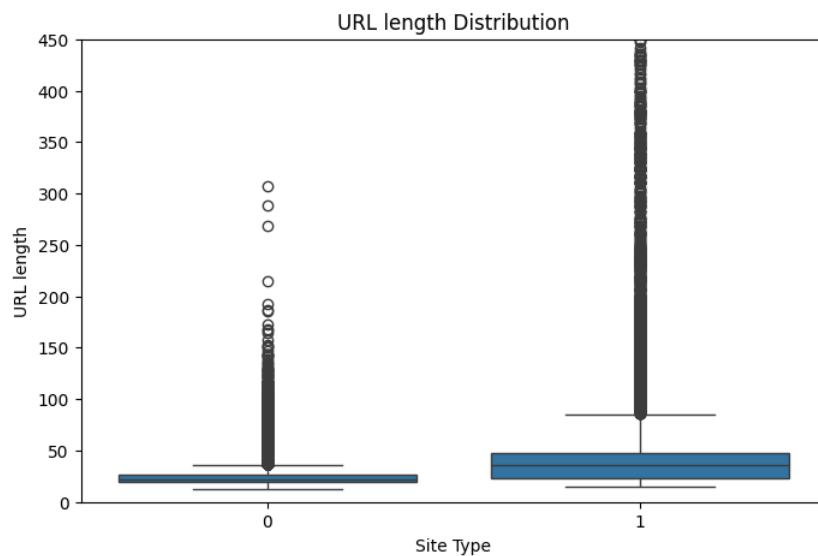
      num_tokens  max_token_len
count  40000.000000      40000.000000
mean    6.425025      16.141375
std     4.282451      24.732778
min     3.000000      5.000000
25%     4.000000      6.000000
50%     5.000000      9.000000
75%     7.000000     16.000000
max    171.000000     1015.000000

```

```

plt.figure(figsize = (8,5))
sns.boxplot(x = "label", y = "url_length", data = df)
plt.title("URL length Distribution")
plt.xlabel("Site Type")
plt.ylabel("URL length")
plt.ylim(0,450)
plt.show()

```



```

# model = XGBClassifier(en_estimators = 200 , max_depth = 8, learning_rate = 0.05, subsample = 0.8, colsample_bytree = 0.8,
#                       eval_metric="logloss", random_state = 42)
# model.fit(X_train, y_train)
# y_pred = model.predict(X_test)
# accuracy = accuracy_score(y_test, y_pred)
# print(f"Accuracy: {accuracy}")

```

Start coding or [generate](#) with AI.

```

from sklearn.model_selection import train_test_split
from xgboost import XGBClassifier, plot_importance
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, roc_auc_score
X = df.drop(["url", "label", "dom", "has_ip"], axis=1)
y = df["label"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
neg = (y_train == 0).sum()
pos = (y_train == 1).sum()
model=XGBClassifier(
    n_estimators=700,
    max_depth=6,
    learning_rate=0.03,
    subsample=0.85,
    colsample_bytree=0.85,
    min_child_weight=3,
    gamma=0.2,

```

```

reg_lambda=1,
eval_metric="logloss",
random_state=42
)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy}")

```

Accuracy: 0.9798125

```

from sklearn.model_selection import StratifiedKFold, cross_val_score
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(model, X, y, cv=cv, scoring="f1")
print(f"CV F1: {scores.mean():.3f} ± {scores.std():.3f}")

```

CV F1: 0.980 ± 0.000

```

print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
print(f"ROC-AUC: {roc_auc_score(y_test, model.predict_proba(X_test)[:,:1]):.4f}")
plot_importance(model, max_num_features=15)
plt.show()

```

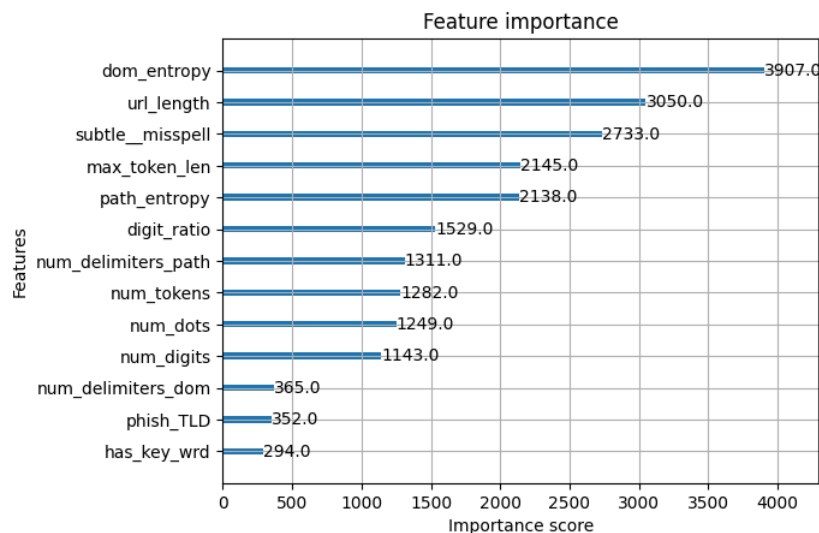
```

[[7822 199]
 [ 124 7855]]

```

	precision	recall	f1-score	support
0	0.98	0.98	0.98	8021
1	0.98	0.98	0.98	7979
accuracy			0.98	16000
macro avg	0.98	0.98	0.98	16000
weighted avg	0.98	0.98	0.98	16000

ROC-AUC: 0.9968



```

import joblib
joblib.dump(model, "phishing_model.pkl")
model = joblib.load("phishing_model.pkl")

```

```

# from sklearn.metrics import f1_score
# import numpy as np
# import pickle
# model = pickle.load(open("phishing_model.pkl", "rb"))
# probs = model.predict_proba(X_test)[: , 1]
# best_threshold = 0
# best_score = -1
# for t in np.arange(0.0, 1.001, 0.001):
#     preds = (probs >= t).astype(int)
#     score = f1_score(y_test, preds)
#     if score > best_score:
#         best_score = score
#         best_threshold = t
# print("Best threshold:", best_threshold)
# print("Best F1:", best_score)

```

Best threshold: 0.405
Best F1: 0.980045999875676

```

def predict_url(url):
    url = url.replace("://www.", "://")
    feat = extract_feat(url, lbl=None)
    feat.pop("url")
    feat.pop("dom")
    feat.pop("label")
    feat.pop("has_key_wrd")

```

```
def predict_url(url):
    df_input = pd.DataFrame([feat])
    prob = model.predict_proba(df_input)[0][1]
    label = "🚩 Phishing" if prob > 0.59 else "✅ Safe"
    return label, float(round(prob, 4))

# Test it
print(predict_url("https://paypal-secure-login.tk/verify/account"))
print(predict_url("https://chatgpt.com/"))
print(predict_url("https://www.google.com/"))
print(predict_url("http://ayareview-document.pdf-iso.webapps-security.review-2jk39w92.glovesforfreemason.com/docusingn/"))
print(predict_url("https://www.youtube.com/watch?v=dQw4w9WgXcQ"))
```

```
('🚩 Phishing', 0.9994000196456909)
('✅ Safe', 0.16509999334812164)
('✅ Safe', 0.5634999871253967)
('🚩 Phishing', 0.8116000294685364)
('✅ Safe', 0.01360000018030405)
```

Start coding or [generate](#) with AI.